

The **Calendar class** in Java represents and manipulates date and time using fields such as YEAR, MONTH, DAY, and HOUR. It is an abstract class that extends Object and implements Comparable, Serializable, and Cloneable, so it cannot be instantiated using a constructor.

- Calendar objects are created using the static Calendar.getInstance() method.
- Calendar.getInstance() returns a concrete Calendar object with the current date and time.
- By default, it uses the system's time zone and locale.
- Overloaded versions of getInstance() allow specifying a custom TimeZone, Locale, or both.

This example demonstrates creating a Calendar object using Calendar.getInstance() and retrieving the current date and time.

```
import java.util.Calendar;

public class GFG {
    public static void main(String[] args) {
        Calendar c= Calendar.getInstance();
        System.out.println("Current date and time: " + c.getTime());
    }
}
```

Output

```
Current date and time: Fri Dec 19 06:28:56 UTC 2025
```

Explanation:

- **Calendar.getInstance()** creates a Calendar object set to the current date and time.
- **getTime()** converts the calendar value into a Date object for display.

Important Calendar Methods

METHOD	DESCRIPTION
add(int field, int amount)	Adds or subtracts a specified amount of time from a calendar field.

METHOD	DESCRIPTION
int get(int field)	Returns the value of a specific calendar field.
getMaximum(int field)	Returns the maximum valid value for a calendar field.
getMinimum(int field)	Returns the minimum valid value for a calendar field.
Date getTime()	Returns a Date object representing calendar time

Below examples illustrate the above methods:

Example 1: add() Method

This example shows how to modify dates using the add() method of the Calendar class.

```
import java.util.*;

public class GFG {
    public static void main(String[] args) {
        Calendar c = Calendar.getInstance();

        c.add(Calendar.DATE, -15);
        System.out.println("15 days ago: " + c.getTime());

        c.add(Calendar.MONTH, 4);
        System.out.println("4 months later: " + c.getTime());

        c.add(Calendar.YEAR, 2);
        System.out.println("2 years later: " + c.getTime());
    }
}
```

Output

15 days ago: Thu Dec 04 07:31:57 UTC 2025

4 months later: Sat Apr 04 07:31:57 UTC 2026

2 years later: Tue Apr 04 07:31:57 UTC 2028

Explanation:

- add(Calendar.DATE, -15) moves the date 15 days backward.
- add(Calendar.MONTH, 4) moves the date 4 months forward.
- add(Calendar.YEAR, 2) moves the date 2 years forward.

Example 2: get() Method

This example retrieves specific date and time fields from the current calendar using the Calendar class.

```
import java.util.*;
```

```
public class GFG {  
    public static void main(String[] args) {  
        Calendar c = Calendar.getInstance();  
  
        System.out.println("Year: " + c.get(Calendar.YEAR));  
        System.out.println("Date: " + c.get(Calendar.DATE));  
        System.out.println("Minute: " + c.get(Calendar.MINUTE));  
        System.out.println("Second: " + c.get(Calendar.SECOND));  
    }  
}
```

Output

Year: 2025

Date: 19

Minute: 18

Second: 42

Explanation:

- get(Calendar.YEAR) returns the year.
- get(Calendar.DATE) returns the day of the month.

- get(Calendar.MINUTE) and get(Calendar.SECOND) return the current minute and second.

Example 3: getMaximum() Method

This example shows how to find the maximum possible values of specific calendar fields using the Calendar class.

```
import java.util.*;

public class GFG {
    public static void main(String[] args) {
        Calendar c = Calendar.getInstance();

        System.out.println("Days in a week: "
+c.getMaximum(Calendar.DAY_OF_WEEK));
        System.out.println("Weeks in a year: "
+c.getMaximum(Calendar.WEEK_OF_YEAR));
    }
}
```

Output

```
Days in a week: 7
Weeks in a year: 53
```

Explanation:

- getMaximum(Calendar.DAY_OF_WEEK) returns the maximum number of days in a week.
- getMaximum(Calendar.WEEK_OF_YEAR) returns the maximum number of weeks in a year.